



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5
з дисципліни «Технології розроблення
програмного забезпечення»
Тема: «Патерни проектування»
Варіант «Mind-mapping software»

Виконав:

студент групи ІА-32

Бурдін Д. Є.

Перевірив:

Мягкий М. Ю.

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та навчитися застосовувати їх в реалізації програмної системи.

Тема проєкту: Mind-mapping software

Патерни: Strategy, Prototype, Abstract Factory, Bridge, Composite, SOA

Опис: Візуальний додаток для складання "карт пам'яті" з можливістю роботи з декількома картами (у вкладках), автоматичного промальовування ліній, додавання вкладених файлів, картинок, відеофайлів (попередній перегляд); можливість додавання значків категорій / терміновості, обведення областей карти (поділ пунктирною лінією)

Зміст

Теоретичні відомості.....	2
Хід роботи	4
Діаграма класів патерну «Prototype»	5
Код програми	5
Висновок	9
Питання до лабораторної роботи	9

Теоретичні відомості

Шаблон «Prototype»

Призначення патерну: Шаблон «Prototype» (Прототип) використовується для створення об'єктів за «шаблоном» (чи «кресленням», «ескізом») шляхом копіювання шаблонного об'єкту, який називається прототипом. Для цього визначається метод «клонувати» в об'єктах цього класу.

Цей шаблон зручно використати, коли заздалегідь відомо як виглядатиме кінцевий об'єкт (мінімізується кількість змін до об'єкту шляхом створення шаблону), а також для видалення необхідності створення об'єкту – створення відбувається за рахунок клонування, і самій програмі немає необхідності знати, як створювати об'єкт.

Також, це дозволяє маніпулювати об'єктами під час виконання програми шляхом налаштування відповідних прототипів; значно зменшується ієрархія наслідування (оскільки в іншому випадку це були б не прототипи, а вкладені класи, що наслідують)

Проблема: Ви розробляєте редактор рівнів для 2D гри на основі спрайтів. В панелі інструментів ви маєте багато кнопок для різних елементів, які можна розташовувати на екрані, такі як сходи, стіни, підлога, оздоблення та інші. Ці елементи у вас об'єднані в ієрархію з базовим класом `GameObject`. Під кожен елемент можна зробити свій тип кнопки, але тоді ми отримаємо паралельну ієрархію кнопок і при додаванні нового типу ігрового об'єкту потрібно буде додавати і новий тип кнопки.

Рішення: Використовуючи патерн прототип, додаємо до базового об'єкта `GameObject` метод `Clone()`, а кнопки будуть зберігати посилання на об'єкт базового типу `GameObject`. При натисканні на кнопку об'єкт який потрібно додати на ігровому полі отримуємо не створенням нового, а клонуванням прототипу, який прив'язаний до кнопки. Таким чином, коли ми будемо додавати нові типи ігрових об'єктів, то логіка роботи з кнопками не буде змінюватися, тому що не має прив'язки до конкретних типів.

Також слід відзначити, що при копіюванні об'єкту, навіть приватні поля будуть скопійовані, тому що реалізація методу копіювання знаходиться в цьому класі, що копіюється і таким чином є доступ для копіювання і до відкритих і до закритих полів.

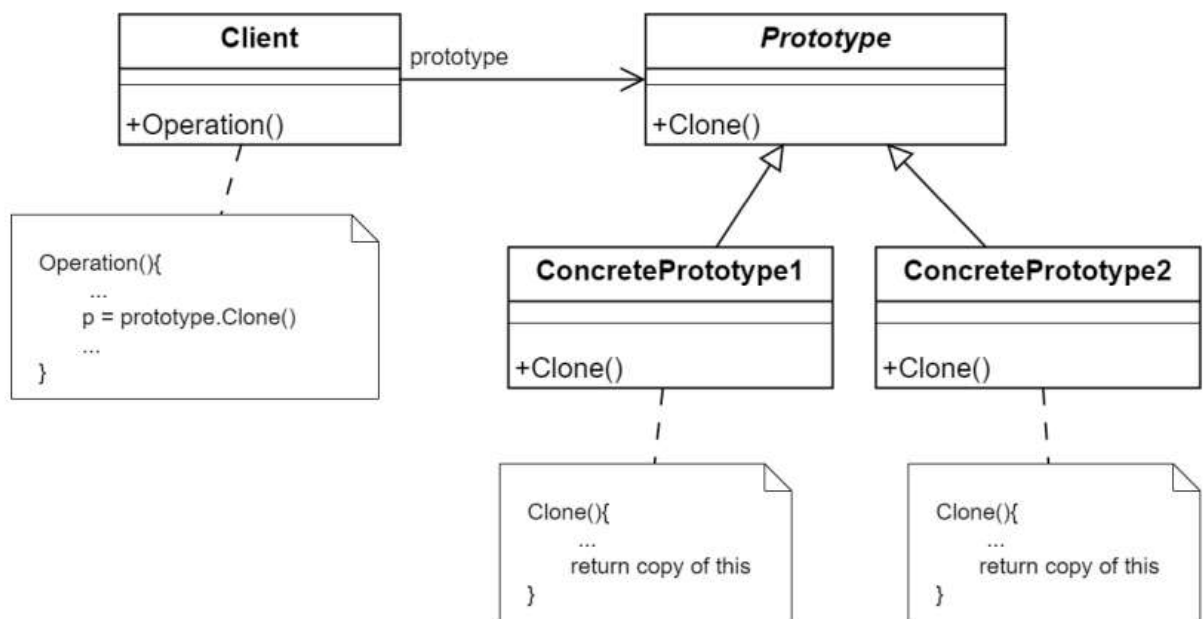


Рис. 1. Структура патерну «Прототип»

Переваги та недоліки:

- + За рахунок клонування складних об'єктів замість їх створення, підвищується продуктивність.
- + Різні варіації об'єктів можна отримувати за рахунок клонування, а не розширення ієрархії класів.
- + Вища гнучкість, тому що клоновані об'єкти можна модифікувати незалежно, не впливаючи на об'єкт з якого була зроблена копія.
- Реалізація глибокого клонування досить проблематична, коли об'єкт що клонується містить складну внутрішню структуру та посилання на інші об'єкти.
- Надмірне використання патерну Прототип може привести до ускладнення коду та проблем із супроводом такого коду.

Хід роботи

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів патерну «Prototype»

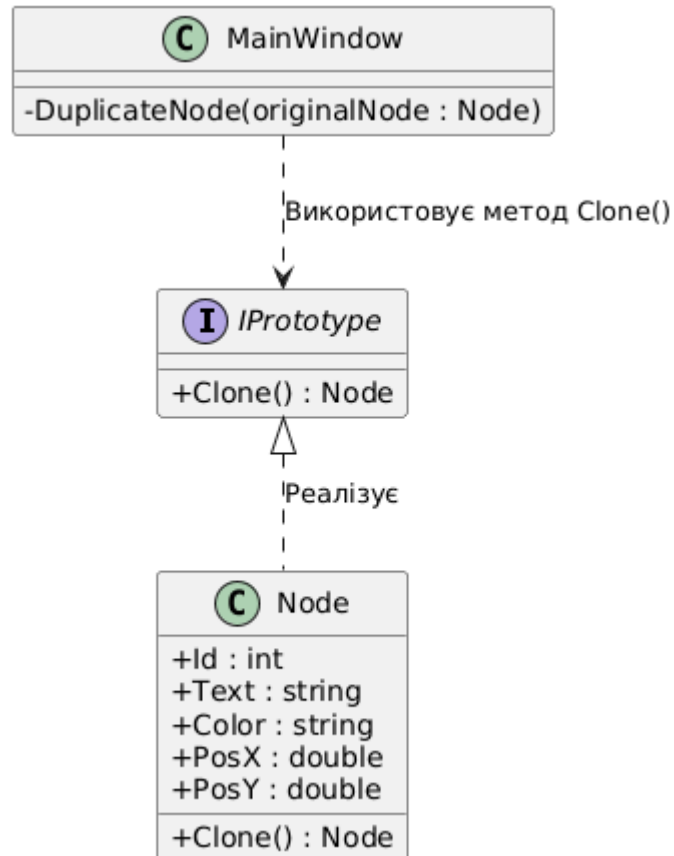


Рис. 2. Діаграма класів патерну Prototype

Код програми

IPrototype (створено)

```
using MindMapApp.Entities;
namespace MindMapApp.Entities
{
    public interface IPrototype
    {
        Node Clone();
    }
}
```

Node (додано метод Clone)

```
public Node Clone()
{
    return new Node
    {
        Id = 0,
        Text = this.Text + " (Копія)",
        Color = this.Color,
        PosX = this.PosX + 20,
        PosY = this.PosY + 20,
        MindMapId = this.MindMapId,
        RegionId = this.RegionId
    };
}
```

MainWindow.xaml.cs (оновлено метод DrawCurrentMap())

```
public void DrawCurrentMap()
{
    DrawingCanvas.Children.Clear();

    if (CurrentMap == null) return;

    if (CurrentMap.Connections != null)
    {
        foreach (var conn in CurrentMap.Connections)
        {
            var fromNode = CurrentMap.Nodes.FirstOrDefault(n => n.Id == conn.FromNodeId);
            var toNode = CurrentMap.Nodes.FirstOrDefault(n => n.Id == conn.ToNodeId);
```

```

if (fromNode != null && toNode != null)
{
    var line = new System.Windows.Shapes.Line
    {
        X1 = fromNode.PosX + 40,
        Y1 = fromNode.PosY + 20,
        X2 = toNode.PosX + 40,
        Y2 = toNode.PosY + 20,
        Stroke = System.Windows.Media.Brushes.Gray,
        StrokeThickness = 2
    };
    DrawingCanvas.Children.Add(line);
}
}
}

```

```

if (CurrentMap.Nodes != null)
{
    foreach (var node in CurrentMap.Nodes)
    {
        var ellipse = new System.Windows.Shapes.Ellipse
        {
            Width = 80,
            Height = 40,
            Fill = (System.Windows.Media.SolidColorBrush)new
System.Windows.Media.BrushConverter().ConvertFrom(node.Color),
            Stroke = System.Windows.Media.Brushes.Black,
            StrokeThickness = 1

```

```
};
```

```
var textBlock = new TextBlock  
{  
    Text = node.Text,  
    HorizontalAlignment = HorizontalAlignment.Center,  
    VerticalAlignment = VerticalAlignment.Center,  
    TextTrimming = TextTrimming.CharacterEllipsis,  
    MaxWidth = 70  
};
```

```
var grid = new Grid { Width = 80, Height = 40, Cursor = Cursors.Hand };  
grid.Children.Add(ellipse);  
grid.Children.Add(textBlock);
```

```
Canvas.SetLeft(grid, node.PosX);  
Canvas.SetTop(grid, node.PosY);  
// контекстне меню  
var contextMenu = new ContextMenu();  
var cloneItem = new MenuItem { Header = "Дублювати" };  
cloneItem.Click += (s, e) =>  
{  
    var clonedNode = node.Clone();  
    CurrentMap.Nodes.Add(clonedNode);  
    Repository.Update(CurrentMap);  
    DrawCurrentMap();  
};
```

```
contextMenu.Items.Add(cloneItem);
```



```

grid.ContextMenu = contextMenu;

grid.MouseLeftButtonDown += (sender, e) =>
{
    e.Handled = true;
    HandleGlobalClick(node);
};

DrawingCanvas.Children.Add(grid);
}
}
}

```

Висновок: В ході виконання лабораторної роботи було реалізовано патерн «Prototype» що забезпечило функціонал створення нових об'єктів шляхом клонування існуючих екземплярів замість створення з нуля

Питання до лабораторної роботи

1. Яке призначення шаблону «Адаптер»?
Для організації взаємодії об'єктів з несумісними інтерфейсами. Він діє як обгортка, трансформуючи інтерфейс одного класу в інтерфейс, який очікує клієнт, дозволяючи їм працювати разом без зміни вихідного коду.
2. Які класи входять в шаблон «Адаптер», та яка між ними взаємодія?
Клієнт, Ціль (Target), Адаптер (реалізує Target і викликає методи Адаптовуваного) та Адаптовуваний (Adaptee). Адаптер отримує виклик від Клієнта і перенаправляє його Адаптовуваному у зрозумілому для нього форматі.
3. Яка різниця між реалізацією «Адаптера» на рівні об'єктів та на рівні класів?
Адаптер об'єктів використовує композицію – зберігає посилання на екземпляр класу Adaptee, що дозволяє адаптувати як сам клас, так і його підкласи. Адаптер класів використовує успадкування (часто множинне): він успадковує інтерфейс Target і код класу Adaptee, що дозволяє перевизначати методи Adaptee, але жорстко прив'язує до конкретного класу.

4. Яке призначення шаблону «Будівельник»?
Призначений для покрокового створення складних об'єктів. Він відокремлює алгоритм конструювання об'єкта від його внутрішнього представлення, дозволяючи використовувати один і той самий процес будівництва для отримання різних варіацій об'єкта.
5. Які класи входять в шаблон «Будівельник», та яка між ними взаємодія?
Директор, Будівельник, Конкретний Будівельник та Продукт (складний об'єкт, що створюється). Директор дає вказівки Будівельнику, а Клієнт забирає готовий Продукт у Будівельника.
6. У яких випадках варто застосовувати шаблон «Будівельник»?"
Коли об'єкт має складну внутрішню структуру або багато опціональних параметрів (уникнення «телескопічного конструктора»), коли процес створення має бути розділений на кілька етапів, або коли потрібно створювати різні представлення одного й того ж об'єкта.
7. Яке призначення шаблону «Команда»?
Перетворення запиту (дію) на об'єкт. Це дозволяє передавати дії як аргументи, ставити їх у чергу, логувати, а також підтримувати скасування (Undo) операцій, відокремлюючи ініціатора запиту від об'єкта, що його виконує.
8. Які класи входять в шаблон «Команда», та яка між ними взаємодія?
Відправник, Команда, Конкретна Команда, Отримувач та Клієнт. Відправник викликає метод виконання у Команди, а вона делегує роботу Отримувачу.
9. Розкажіть як працює шаблон «Команда».
Клієнт створює об'єкт Конкретної Команди, налаштовуючи його на потрібного Отримувача. Цей об'єкт передається Відправнику (наприклад, кнопці).
10. Яке призначення шаблону «Прототип»?
Для створення нових об'єктів шляхом копіювання вже існуючих екземплярів-прототипів
11. Які класи входять в шаблон «Прототип», та яка між ними взаємодія?
Інтерфейс Прототипу, Конкретний Прототип та Клієнт. Клієнт замість створення об'єкта через new викликає метод клонування у наявного об'єкта, отримуючи його точну копію.
12. Які можна привести приклади використання шаблону «Ланцюжок відповідальності»?
Приклади: система обробки подій у UI (клік передається від кнопки до контейнера, а потім до вікна), middleware у веб-фреймворках (послідовна перевірка авторизації, логування, валідація даних перед обробкою запиту)